



DEEP LEARNING

AULA 4



Prof. Marco Teixeira



CONVERSA INICIAL

Até agora, sabemos um pouco sobre como as técnicas de aprendizado de máquina funcionam, rodando exemplos prontos, e criando e treinando uma Rede Neural Artificial (RNA) para classificar letras de um alfabeto. As RNAs são capazes de resolver vários problemas de classificação, no entanto, a sua complexidade aumenta com o aumento de números de neurônios de entrada. Quanto mais dados na entrada da rede, maior será a quantidade de neurônios a serem treinados, e com isso maior será o poder computacional necessário. Uma alternativa às RNAs é o uso de Redes Neurais Convolucionais, tipicamente conhecidas como *CNN*, do Inglês *Convolutional Neural Network*. Essas redes são capazes de extrair características dos dados, antes de incluí-los em um classificador, como um MLP. Essas redes também exigem um poder de processamento na hora de realizar o treino, no entanto diminuem em muito o número de neurônios necessários para a criação e classificação de elementos. Nesta etapa, iremos estudar um pouco melhor os conceitos de CNN, e também realizar um exemplo prático.

Ao final desta etapa, esperamos atingir os seguintes objetivos que serão avaliados ao longo do estudo da forma indicada.

Objetivos
1 – Entender os conceitos de uma <i>Convolutional Neural Networ (CNN)</i>
2 – Conhecer os modelos de camadas existentes na CNN
3 – Realizar um experimento prático

TEMA 1 – Conceito de *Convolutional Neural Network (CNN)*

1.1 Introdução

Proposta pelo pesquisador francês Yann LeCun, é uma variação das tradicionais Redes Neurais Artificiais (RNA), ou do inglês *Artificial Neural Network (ANN)*. São muito utilizadas para problemas de classificação, principalmente em imagens. Do mesmo modo que vimos no estudo de RNA, as CNNs utilizam um conjunto de dados de entrada e saída, e a partir desses dados



a rede é treinada, para posteriormente realizar a classificação com dados não vistos pela rede.

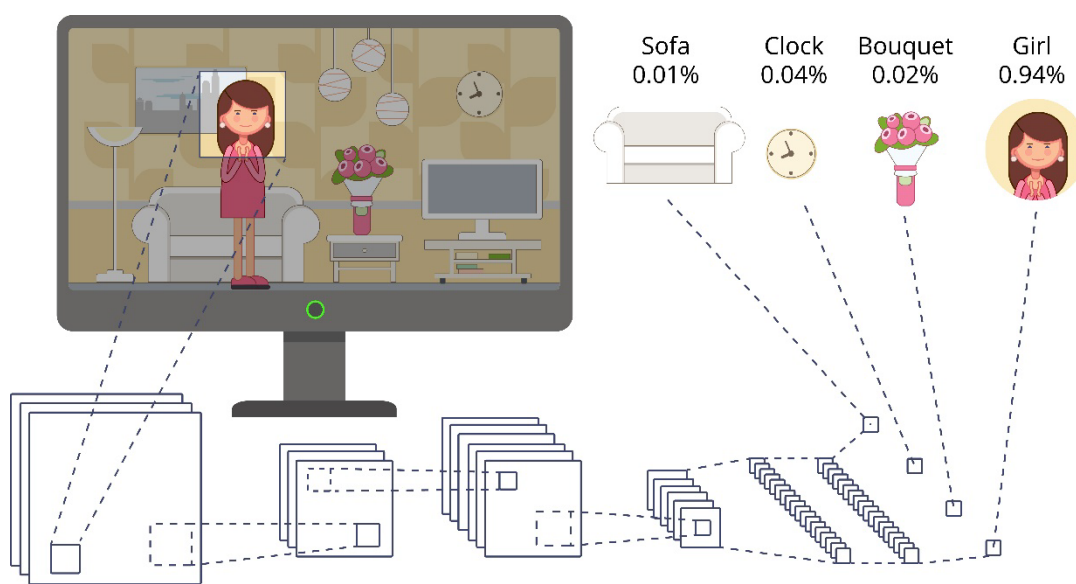
Uma das principais diferenças entre a CNN e uma RNA são suas camadas. As RNAs são compostas, em sua maioria, por camadas totalmente conectadas, enquanto as CNNs são compostas por diferentes tipos de camadas, sendo as mais comuns: Camada de convolução, camada de *pooling*, e rede totalmente conectada, como vimos em RNAs.

Mas porque a CNN utiliza tantas camadas?

As camadas anteriores à rede, totalmente conectada, são utilizadas para extrair características das imagens, comumente chamados de *features*. O classificador conectado ao fim da rede (rede totalmente conectada) necessita classificar essas características, e não a imagem toda. Desta forma, a rede se torna mais leve, não precisando de um grande volume de neurônios, como aconteceria se fosse adicionado cada pixel da imagem original a um neurônio em uma rede totalmente conectada.

No entanto, para tirar essas *features*, as camadas de convolução e pooling também necessitam ser treinadas. Desta forma, uma CNN pode possuir menos neurônios que uma RNA, no entanto o seu treinamento pode ser mais complexo e exigir mais tempo e poder de processamento. A Figura 1 apresenta um exemplo de uma *Convolutional Neural Network*, com suas diferentes camadas.

Figura 1 – Exemplo de uma arquitetura de uma rede CNN



Créditos: Arkadiy Chumakov/Shutterstock.



Como visto, uma CNN é formada por diferentes tipos de camadas. Nesta seção, iremos estudar um pouco sobre os principais tipos de camadas existentes em uma CNN, começando pela camada convolucional.

TEMA 2 – Camada convolucional

A camada convolucional é responsável por extrair algumas informações da imagem. É aplicado um filtro, chamado de *máscara* ou *kernel*, com tamanho definido por quem está projetando/implementando a CNN, que irá percorrer toda a imagem original e criará uma nova tabela/imagem com dados extraídos da imagem. Esse *kernel* é inicializado com valores aleatórios, e então, durante o treinamento, seus valores são ajustados, até chegar a um bom resultado.

As camadas convolucionais agem de maneira similar a filtros tradicionais em visão computacional, como filtros de detecção de contorno, como por exemplo o filtro de Sobel, que consiste em aplicar máscaras (kernel) por toda a imagem, e como resultado possuem uma nova imagem, com contornos destacados. Os exemplos de Kernel de Sobel podem ser vistos na Figura 2.

Figura 2 – Exemplo de máscara (Kernel) que pode ser aplicado a uma imagem digital. Neste caso, se trata das máscaras do filtro de Sobel

-1	0	1
-2	0	2
-1	0	1

1	2	1
0	0	0
-1	-2	1

Escrita: [-1 0 1; -2 0 2; -1 0 1] [1 2 1; 0 0 0; -1 -2 -1]

No entanto, a máscara de Sobel é fixa, e faz parte das técnicas tradicionais de visão computacional, enquanto a camada convolucional é treinada durante a etapa de treinamento da CNN. Mas o funcionamento das duas técnicas é semelhante. Imagine uma imagem, que consiste em uma matriz podendo ter diferentes dimensões. O filtro então desliza pela imagem, onde cada



pixel da imagem estará sobre um pixel no filtro. Então será multiplicado o valor do pixel pelo valor do filtro, e o resultado de todas as multiplicações será somada, e então teremos o novo valor do pixel na nova imagem, como pode ser visto na Figura 3. Neste exemplo, temos $-1 \times 100 + -2 \times 100 + -1 \times 100 + 0 \times 100 + 0 \times 100 + 0 \times 100 + 1 \times 200 + 2 \times 200 + 1 \times 200$, resultando no valor do novo pixel em 400. A aplicar esse cálculo, o filtro se desloca para baixo ou para o lado, e então é executado novamente os mesmos passos, dando o novo valor para o pixel da nova imagem.

Figura 3 – Exemplo de aplicação de um mascara/kernel em uma imagem

100	100	200	200
100	100	200	200
100	100	200	200
100	100	200	200

-1	0	1
-2	0	2
-1	0	1

-100
-200
-100
200
400
<u>+200</u>
=400

2.1 Salto (stride)

A camada convolucional pode percorrer toda a imagem, como acontece em técnicas tradicionais de visão computacional, ou pode realizar saltos (stride). Quando são realizados saltos, a matriz de características se torna menor, o que em alguns casos é desejável, ou não, dependendo de muitos fatores, como por exemplo a quantidade e qualidade dos dados a serem classificados.

Desta forma, o stride pode ser definido ao criar uma CNN, seu resultado é o pulo do kernel ao ser aplicado na imagem original. Caso tenha um stride maior que 1, será desconsiderada uma fileira/coluna da imagem original, onde o kernel pulará, e não terá um “output” referente a essa fileira/coluna.

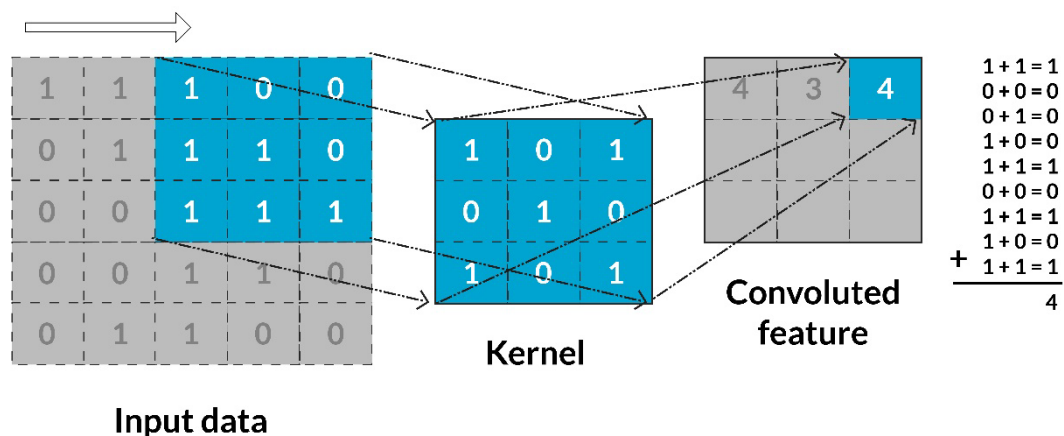


2.2 Preenchimento (padding)

Outro parâmetro que deve ser preenchido pelo desenvolvedor é o *padding*. Basicamente consiste em adicionar bordas na imagem, preenchidas com zero, para que o kernel possa percorrer até mesmo os pixels das bordas, e dessa forma evitar a perda de dados, que podem ser importantes para a classificação.

Imagine a seguinte situação, o kernel precisa estar sobre os pixels de uma imagem. O kernel pode ter tamanhos variados, podendo ser pequeno ou grande. Quando chega ao canto da imagem, não é possível deixar o meio do Kernel sobre os pixels de borda, o que ocasiona a perda desses pixels, como mostrado na Figura 4, onde é possível verificar que o resultado é uma imagem menor, contendo somente as partes onde o kernel pode percorrer.

Figura 4 – Exemplo de aplicação de uma máscara em uma imagem, sem a utilização de *padding*



No entanto, quando é utilizado o *padding*, é possível adicionar bordas com valores zero (0) ao redor da imagem, e desta forma o filtro pode percorrer todos os pixels da imagem. Como mostrado na Figura 5.



Figura 5 – Exemplo de aplicação de uma máscara em uma imagem, com a utilização de *padding*

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

O resultado da camada convolucional é adicionado a uma função de ativação. Como visto no estudo sobre RNA, existem diferentes funções de ativação, como sigmoid, tanh, softmax e Relu. Em camadas convolucionais, é comum o uso da função de ativação Relu. Basicamente, essa função irá zerar os valores negativos obtidos pela etapa de convolução.

TEMA 3 – CAMADA DE POOLING

O *pooling* consiste em uma estratégia para diminuir a quantidade de características da camada anterior. Do mesmo modo que na camada convolucional, possui um tamanho definido, como por exemplo 2x2, que irá ser aplicado em todos os pixels da imagem anterior, e então teremos uma nova imagem ou matriz de dados (matriz de dados, pois o resultado em alguns casos não é entendível pelo humano, mas seus padrões são reconhecidos por uma CNN).

O pulo do pooling é relacionado a seu tamanho. Como a sua finalidade é diminuir a quantidade de dados, ele não passa por toda a imagem, como acontece coma camada convolucional. Por exemplo, um polling de tamanho 2x2, em uma imagem de tamanho 48x48, por exemplo, terá como resultado uma nova imagem/dados de tamanho de 24x24.

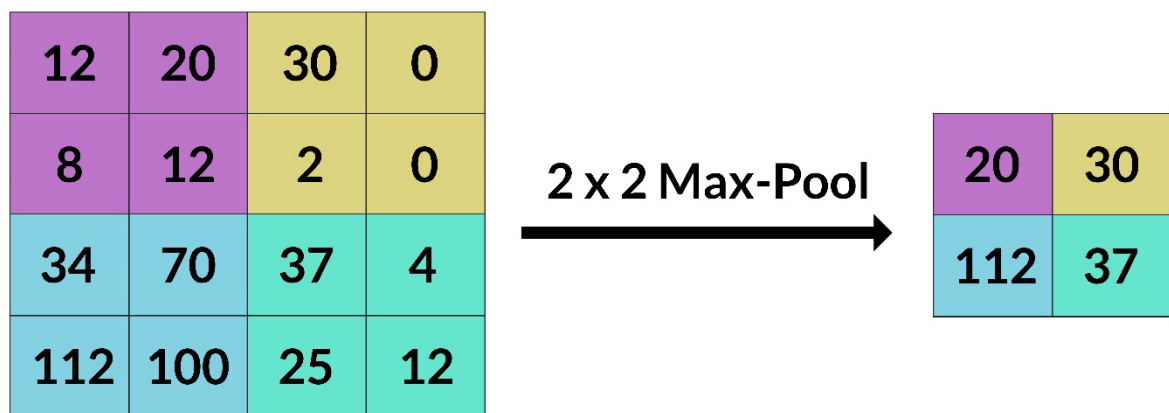
Existem diferentes estratégias no pooling, para definir qual valor deve prosseguir para a próxima camada. A seguir, iremos estudar alguns deles.



3.1 Max pooling

O Maxpooling é responsável por selecionar o maior valor dentro os pixels que estão sendo analisados pelo kernel, como apresentado na Figura 6. Neste exemplo, um kernel de tamanho 2x2 percorreu toda a imagem de tamanho 4x4. Em cada lugar que o kernel passou, ele verificou qual é o maior valor entre os dados ali existentes, o escolhendo. Como resultado, a matriz de tamanho 4x4 foi reduzido para uma matriz de tamanho 2x2.

Figura 6 – Exemplo de funcionamento da camada *Max pooling*

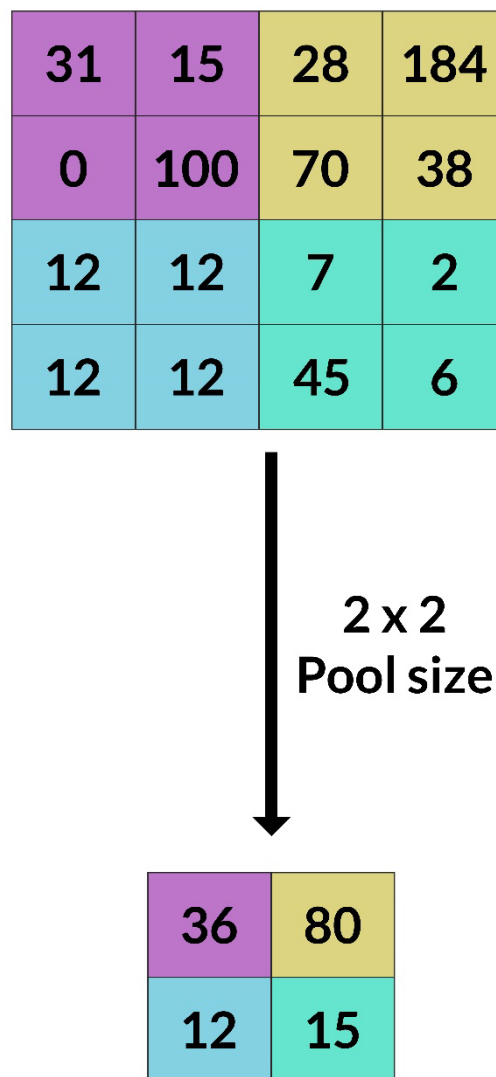


3.1 Average Pooling

Outra forma de agrupar os dados, é pela média. Conhecido como *Average Pooling*, consiste em agrupar os dados pela média, de acordo com o tamanho do kernel. Por exemplo, um *pooling* de 2x2 que utiliza a média, irá realizar a somatória de todos os valores na matriz dentro do kernel, e então irá dividi-los por 4, criando assim a média dos valores. Um exemplo pode ser visto na Figura 7.



Figura 7 – Exemplo de funcionamento da camada *Average Pooling*



TEMA 4 – VAMOS PRATICAR

Nada melhor do que praticarmos, para então podermos entender o que de fato está sendo desenvolvido. O exemplo que iremos executar a seguir pode ser encontrado na página oficial do Keras, pelo link: <https://keras.io/examples/generative/pixelcnn/> (Acesso em: 18 ago. 2022). Esse exemplo é rápido e fácil de ser seguido, com conjunto de dados fornecido pela própria ferramenta. Iremos então realizar o exemplo, entender o código e os resultados obtidos.



4.1 Importando as bibliotecas necessários

Primeiramente, vamos importar as bibliotecas necessárias para que possamos então criar e utilizar a nossa rede. Para isso, basta adicionar ao código as linhas presentes no Código 1.

Código 01: Importação das bibliotecas necessárias para a criação de nossa CNN.

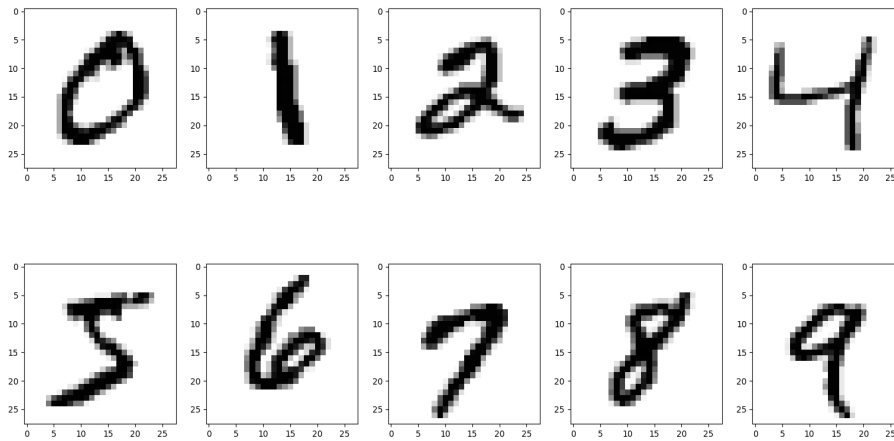
```
-----  
1. import numpy as np  
2. import tensorflow as tf  
3. from tensorflow import keras  
4. from tensorflow.keras import layers  
5. from tqdm import tqdm  
6. import matplotlib.pyplot as plt  
-----
```

Fonte: Teixeira, 2022.

4.2 Criando o conjunto de dados

Como mencionado, iremos utilizar um conjunto de dados, contendo a imagem de entrada, junto com o seu *label*, ou seja, junto com o resultado esperado para a classificação. Esse conjunto de dados é fornecido pela própria ferramenta, sendo ele o mnist. Um exemplo dos dados presentes pode ser visto na Figura 8. Consiste basicamente em números escritos de 0 a 9

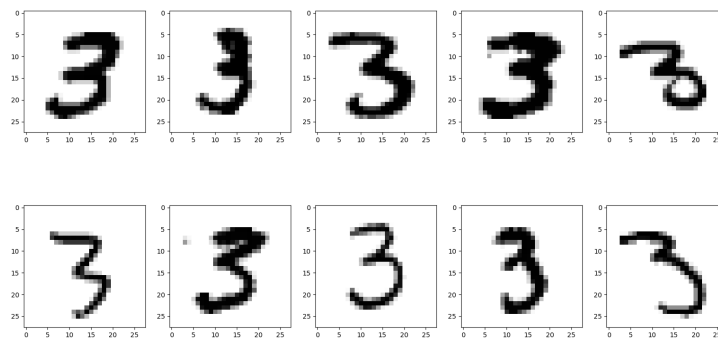
Figura 8 – Exemplo de dados existentes no conjunto de dados mnist.



Fonte: Teixeira, 2022.

Vale ressaltar que os números são escritos de forma diferente, ou seja, não são somente estas 9 imagens. O conjunto é composto por diversas imagens, neste exemplo, iremos utilizar 60000 imagens para treino e 10000 para testes. A Figura 9 apresenta um pouco as diferenças existentes entre as imagens da mesma classe.

Figura 9 – Exemplo dos diferentes tipos de dados da mesma classe presentes no mnist.



Fonte: Teixeira, 2022.

Vamos ao código! O Código 2 apresenta as configurações iniciais. Temos, neste conjunto de dados, 10 classes, sendo cada classe um número de 0 a 10. Na linha 2 é apresentada a configuração do formato da imagem de entrada. No caso é uma imagem binária de 28x28, conforme visto das Figuras X e X. Repare que este é um exemplo puramente acadêmico! No mundo real, lidaremos com imagens coloridas, de tamanho cada vez maiores, o que acarreta aumento considerável no tempo de treinamento e execução de uma CNN. No entanto, os passos para criação e treinamento da rede são os mesmos, o que não justifica



sacrificarmos dias para treinar uma CNN, se podemos fazer em minutos com o uso da Mnist.

Código 2: Configuração de número de classes e formato das imagens de entrada em nossa CNN.

```
-----  
  
1. num_classes = 10  
  
2. formato_entrada = (28, 28, 1)  
  
-----
```

Fonte: Teixeira, 2022.

Criando os dados, é possível carregar os dados. Como os dados mnist são do teste padrão do Keras, ele mesmo realiza a separação dos dados para nós, entre conjunto de teste e treino. O Código 3 apresenta a linha de comando necessário para tal tarefa.

Código 3: Carregando os dados do mnist em variáveis de treino e teste.

```
-----  
  
1. (x_treino, y_treino), (x_teste, y_teste) = keras.datasets.mnist.load_data()  
  
2. #Verificando as dimensões dos dados  
  
3. print("Dimensão de x_treino:", x_treino.shape)  
  
4. print(x_treino.shape[0], "treino ")  
  
5. print(x_teste.shape[0], "teste")  
  
-----
```

Fonte: Teixeira, 2022.

A linha 3 apresenta a quantidade e dimensão dos dados. É possível observar que existem 60000 dados na variável treino, com dimensão de 28x28 pixels. Já as linhas 4 e 5 apresentam as quantidades de dados para o treino (60000) e teste (10000).



O próximo passo consiste em arrumar o *label*, ou seja, os dados que dizem qual número está em cada imagem, neste nosso exemplo. O valor padrão de “y_treino” e “y_teste” é um inteiro, de 0 a 10, que representa o valor que está na imagem naquela mesma posição. Desta forma, na posição x_treino[1] pode conter uma imagem com o número 5, e na posição y_treino[1] terá um inteiro com o valor 5.

No entanto, para que seja possível treinar uma CNN, precisamos saber em qual classe de saída está ativada. Desta forma, precisamos converter o número 5 presente no y_treino, para um vetor que representa que o quinto neurônio está ativo, sendo algo como: “[0,0,0,0,0,1,0,0,0,0]”. Veja que ativamos o sexto neurônio, isso ocorre, pois, o Mnist começa do número zero, desta forma, o primeiro neurônio representa o número zero, e não o número um. O Código 4 apresenta como realizar essa conversão de forma automática com o uso do Keras.

Código 4: Conversão dos dados de saída, para o treino e validação dos dados.

1. `##Convertendo a saída de único número inteiro para vetor de ativação`
2. `y_treino = keras.utils.to_categorical(y_treino, num_classes)`
3. `y_teste = keras.utils.to_categorical(y_teste, num_classes)`

Fonte: Teixeira, 2022.

Os dados já estão prontos para que possamos criar a nossa CNN e então realizar o treinamento. Com a finalidade visual, vamos construir o Código 5. Esse código tem por finalidade exibir 10 exemplos de cada uma das classes. Caso não queira que seja exibido toda vez que o código for executado, basta configurar a variável “exibir_dados”, presente na linha 2, com o valor 0. Aconselho e verificarem pelo menos uma vez os dados, para que fique claro em sua mente o que desejamos classificar.

Código 5: Código para gerar imagens a partir dos dados de treino.

1. `# Exibe 10 exemplos de todas as 10 classes`



```
2.  exhibir_dados = 1

3.  if exhibir_dados ==1:

4.      for i in range(0,10):

5.          img = []

6.          for j in range(0,len(y_treino)):

7.              if y_treino[j][i] == 1:

8.                  print(y_treino[j])

9.                  img.append(x_treino[j].reshape(28,28))

10.                 if (len(img) >= 10):

11.                     break

12.                 if (len(img) >= 10):

13.                     fig, ax = plt.subplots(2,5)

14.                     ax[0,0].imshow(img[0].reshape(28,28),
cmap=plt.cm.binary)

15.                     ax[0,1].imshow(img[1].reshape(28,28),
cmap=plt.cm.binary)

16.                     ax[0,2].imshow(img[2].reshape(28,28),
cmap=plt.cm.binary)

17.                     ax[0,3].imshow(img[3].reshape(28,28),
cmap=plt.cm.binary)

18.                     ax[0,4].imshow(img[4].reshape(28,28),
cmap=plt.cm.binary)

19.                     ax[1,0].imshow(img[5].reshape(28,28),
cmap=plt.cm.binary)

20.                     ax[1,1].imshow(img[6].reshape(28,28),
cmap=plt.cm.binary)
```



```
21.         ax[1,2].imshow(img[7].reshape(28,28),
           cmap=plt.cm.binary)

22.         ax[1,3].imshow(img[8].reshape(28,28),
           cmap=plt.cm.binary)

23.         ax[1,4].imshow(img[9].reshape(28,28),
           cmap=plt.cm.binary)

24.         plt.show()
```

Fonte: Teixeira, 2022.

4.3 Criando a nossa CNN

Com os dados prontos, podemos agora criar a nossa CNN. Para isso, iremos utilizar o comando “`keras.Sequential`” e então definir a estrutura de nossa rede. O Código 6 apresenta o código necessário. Observe que na linha 3 está sendo definido o formato da imagem a ser utilizada pela rede. Já na linha 4 estamos criando uma camada convolucional, com tamanho de *kernel* de 3x3 e função de ativação *Relu*. A linha 5 aplica o *Max Polling*, de tamanho 2x2. Posteriormente, na linha 6 é aplicada mais uma camada convolucional e por fim mais um *Max Polling* na linha 7. A linha 8 converte a saída em um vetor para que possa ser adicionado a uma rede totalmente conectada (RNA). A linha 9 define um parâmetro de “*Dropout*”, ou seja, durante o treino, alguns neurônios são desligados, para que a rede possa ser genérica o suficiente.

Código 6: Criação do modelo de nossa CNN.

```
-----

1. model = keras.Sequential(

2.     [

3.         keras.Input(shape=formato_entrada),

4.         layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),

5.         layers.MaxPooling2D(pool_size=(2, 2)),
```

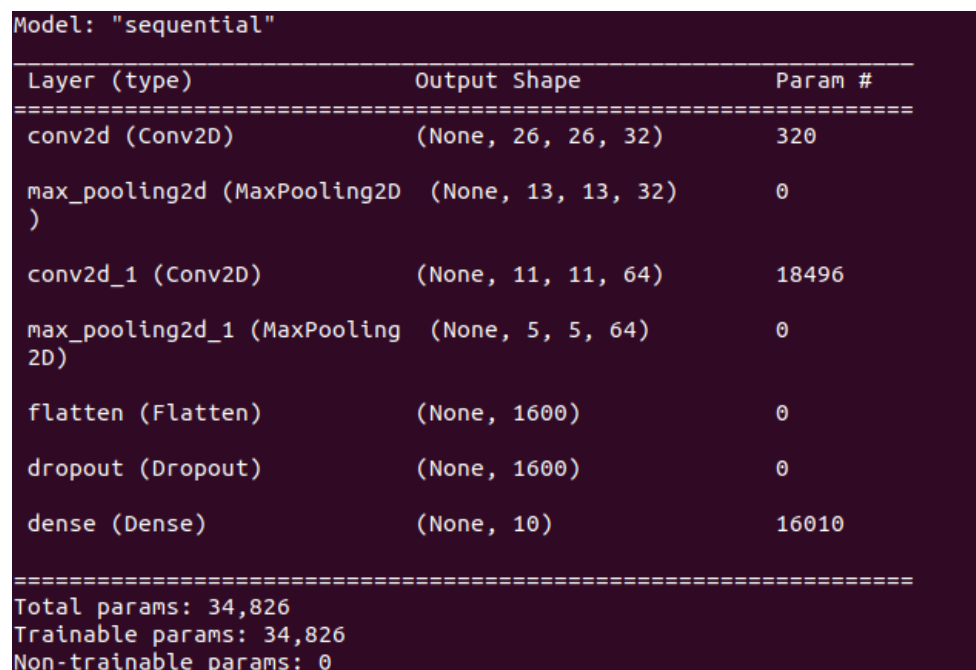


```
6.     layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
7.     layers.MaxPooling2D(pool_size=(2, 2)),
8.     layers.Flatten(),
9.     layers.Dropout(0.5),
10.    layers.Dense(num_classes, activation="softmax"),
11. ]
12. )
13. model.summary()
```

Fonte: Teixeira, 2022.

A linha 13 apresenta a rede criada, vamos observar sua saída, presente na Figura 10.

Figura 10 – Configuração da CNN criada, com todas as suas camadas



Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dropout (Dropout)	(None, 1600)	0
dense (Dense)	(None, 10)	16010
Total params: 34,826		
Trainable params: 34,826		
Non-trainable params: 0		

Fonte: Teixeira, 2022.

4.4 Treinamento a CNN

Com a rede criada, podemos então dar início ao seu treinamento. Primeiro, temos que definir alguns parâmetros, como o tamanho do *batch* e a



quantidade de épocas. Vamos utilizar os valores padrão do exercício, com tamanho de *batch* de 128, ou seja, em cada época serão utilizados 128 dados para o treinamento. E vamos treinar por 15 épocas. Ou seja, será feito 15 rodadas de treinamento, cada uma com 128 dados. Essas configurações podem ser vistas nas linhas 1 e 2 do Código 7.

Código 7: Realizando o treinamento de nossa CNN.

1. tamanho_batch = 128
2. epocas = 15
3. model.compile(loss="categorical_crossentropy",optimizer="adam",
metrics=["accuracy"])
4. model.fit(x_treino, y_treino, batch_size=tamanho_batch, epochs=epocas,
validation_split=0.1)

Fonte: Teixeira, 2022.

Continuando a análise do Código 7, o próximo passo é definir a função de perda, o algoritmo de otimização e a métrica de validação para o algoritmo de treinamento. Neste exemplo, foi definido a função de perda como "categorical_crossentropy", o algoritmo de otimização foi o "adam" e a métrica foi a acurácia.

Feito esse passo, podemos então realizar o treinamento de nossa CNN pelo comando "model.fit", onde definimos os dados de treinamento, o tamanho do batch e a quantidade de épocas, assim como o "validation_split", que consiste na quantidade de dados que será utilizado durante o treinamento para a validação, e assim gerar os dados de acurácia durante o treino. Esse valor deve ser de 0 a 1. Nesse caso foi utilizado 0.1, representando 10% dos dados de treinamento.

4.5 Teste dos resultados

Agora podemos validar os resultados obtidos após o treino. O código 8 realiza esta tarefa, com base no conjunto de dados que separamos para teste no começo desta atividade.



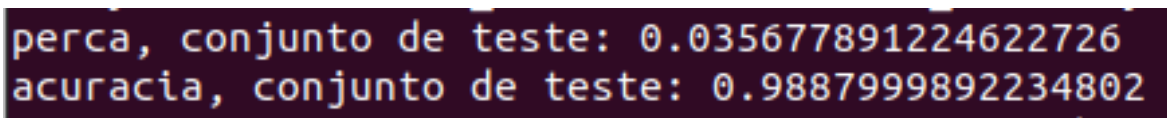
Código 8: Realizando a análise de nosso modelo CNN.

1. resultados = model.evaluate(x_teste, y_teste, verbose=0)
2. print("perca, conjunto de teste:", resultados[0])
3. print("acuracia, conjunto de teste:", resultados[1])

Fonte: Teixeira, 2022.

A função “model.evaluate” valida a nossa rede, que neste momento consiste na variável “model”. Basta adicionar as variáveis de entrada, e a respectiva saída, para que a biblioteca possa conferir se a CNN acertou, ou não, e então gerar a acurácia. O resultado obtido durante esse experimento é apresentado na Figura 11.

Figura 11 – Resultado obtido após treinamento da rede



```
perca, conjunto de teste: 0.035677891224622726
acuracia, conjunto de teste: 0.9887999892234802
```

Fonte: Teixeira, 2022.

4.6 Salvando a rede treinada

Criar e treinar uma CNN é muito importante. Mas o que pode ser mais importante do que criar e treinar uma rede, é salvar a rede para que seja possível utilizá-la futuramente em códigos futuros. Para isso, basta usar a função “model.save”, como apresentado no Código 9.

Código 9: Salvando o nosso modelo.

1. model.save("exemploCNN")

Será criada uma pasta, com o nome “exemploCNN”, e nela será criado vários arquivos necessários para podermos carregar o modelo e aplicá-lo a nossos dados.



4.7 Carregando o modelo salvo

Vamos modificar o código criado até aqui, para podermos carregar o modelo salvo em memória e então aplicá-los aos dados do Mnist. Vamos manter a parte referente ao carregamento dos dados, como presente no Código 10.

Código 10: Carregando os dados para validar o modelo salvo.

```
-----
1. import numpy as np
2. from tensorflow import keras
3. from tensorflow.keras import layers
4. import matplotlib.pyplot as plt
5.
6.
7. num_classes = 10
8. formato_entrada = (28, 28, 1)
9.
10. # Carregando e separando os dados entre treino e teste
11. (x_treino, y_treino), (x_teste, y_teste) = keras.datasets.mnist.load_data()
12.
13.
14. #Verificando as dimensões dos dados
15. print("Dimensão de x_treino:", x_treino.shape)
16. print(x_treino.shape[0], "treino")
17. print(x_teste.shape[0], "teste")
18.
19.
20. ##Convertendo a saída de único número inteiro para vetor de ativação
21. y_treino = keras.utils.to_categorical(y_treino, num_classes)
22. y_teste = keras.utils.to_categorical(y_teste, num_classes)
-----
```

Fonte: Teixeira, 2022.

Agora, no lugar de criarmos e treinarmos um modelo, iremos carregá-lo da memória. Para isso, iremos utilizar o comando “keras.models.load_model”, e



então iremos submeter o modelo carregado da memória pelo mesmo teste aplicado ao modelo treinado, conforme apresentado no Código 11.

Código 11: Carregando o modelo salvo.

```
1. #Carrega modelo da memória
2. model = keras.models.load_model("exemploCNN")
3.
4. model.summary()
5.
6. #Verifica a perda e a acurácia
7. resultados = model.evaluate(x_teste, y_teste, verbose=0)
8. print("perda, conjunto de teste:", resultados[0])
9. print("acuracia, conjunto de teste:", resultados[1])
```

Fonte: Teixeira, 2022.

É possível verificar que agora o modelo não foi treinado novamente, pois o mesmo já estava treinado e salvo na memória, pronto para ser aplicado. O resultado do Código 11 pode ser visto na Figura 12.

Figura 12 – Dados da rede, após ser carregada

Model: "sequential"		
Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dropout (Dropout)	(None, 1600)	0
dense (Dense)	(None, 10)	16010
=====		
Total params: 34,826		
Trainable params: 34,826		
Non-trainable params: 0		
=====		
perda, conjunto de teste: 0.035677891224622726		
acuracia, conjunto de teste: 0.9887999892234802		

Fonte: Teixeira, 2022.



É muito importante ser capaz de utilizar os modelos treinados, pois, caso contrário, por que treinaríamos estes modelos? A etapa de treino pode ser muito demorada, dependendo do equipamento utilizado. Em imagens grandes, pode chegar a dias, semanas e até meses! Já a utilização do modelo treinado é rápida. Após carregado na memória, pode ser chamado a qualquer momento e aplicado nas mais diversas tarefas, como reconhecimento fácil, identificação de objetos em uma foto, entre outras funções

TEMA 5 – CÓDIGOS UTILIZADOS DURANTE A AULA

Nesta seção, vamos deixar os dois códigos utilizados até aqui. O Código 12 representa o código de criação e treino de nossa CNN, enquanto o Código 13 apresenta o carregamento de um modelo salvo, e sua utilização.

5.1 Criando e treinando nossa CNN

Código 12: Criação e treinamento de nossa CNN, com os dados mnist.

```
-----
1. import numpy as np
2. from tensorflow import keras
3. from tensorflow.keras import layers
4. import matplotlib.pyplot as plt

5. num_classes = 10
6. formato_entrada = (28, 28, 1)

7. # Carregando e separando os dados entre treino e teste
8. (x_treino, y_treino), (x_teste, y_teste) = keras.datasets.mnist.load_data()

9. #Verificando as dimensões dos dados
10. print("Dimensão de x_treino:", x_treino.shape)
11. print(x_treino.shape[0], "treino")
12. print(x_teste.shape[0], "teste")

13. ##Convertendo a saída de único número inteiro para vetor de ativação
14. y_treino = keras.utils.to_categorical(y_treino, num_classes)
```



```
15.y_teste = keras.utils.to_categorical(y_teste, num_classes)

16.# Exibe 10 exemplos de todas as 10 classes
17.exibir_dados = 0
18.if exibir_dados ==1:
19.    for i in range(0,10):
20.        img = []
21.        for j in range(0,len(y_treino)):
22.            if y_treino[j][i] == 1:
23.                print(y_treino[j])
24.                img.append(x_treino[j].reshape(28,28))
25.                if (len(img) >= 10):
26.                    break
27.            if (len(img) >= 10):
28.                fig, ax = plt.subplots(2,5)
29.                ax[0,0].imshow(img[0].reshape(28,28),
30.                    cmap=plt.cm.binary)
31.                ax[0,1].imshow(img[1].reshape(28,28),
32.                    cmap=plt.cm.binary)
33.                ax[0,2].imshow(img[2].reshape(28,28),
34.                    cmap=plt.cm.binary)
35.                ax[0,3].imshow(img[3].reshape(28,28),
36.                    cmap=plt.cm.binary)
37.                ax[0,4].imshow(img[4].reshape(28,28),
38.                    cmap=plt.cm.binary)
39.                ax[1,0].imshow(img[5].reshape(28,28),
40.                    cmap=plt.cm.binary)
41.                ax[1,1].imshow(img[6].reshape(28,28),
42.                    cmap=plt.cm.binary)
43.                ax[1,2].imshow(img[7].reshape(28,28),
44.                    cmap=plt.cm.binary)
45.                ax[1,3].imshow(img[8].reshape(28,28),
46.                    cmap=plt.cm.binary)
47.                ax[1,4].imshow(img[9].reshape(28,28),
48.                    cmap=plt.cm.binary)
```



```
39. plt.show()

40. model = keras.Sequential(
41.     [
42.         keras.Input(shape=formato_entrada),
43.         layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
44.         layers.MaxPooling2D(pool_size=(2, 2)),
45.         layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
46.         layers.MaxPooling2D(pool_size=(2, 2)),
47.         layers.Flatten(),
48.         layers.Dropout(0.5),
49.         layers.Dense(num_classes, activation="softmax"),
50.     ]
51.)

52. model.summary()

53. tamanho_batch = 128
54. epocas = 15
55. model.compile(loss="categorical_crossentropy", optimizer="adam",
56.               metrics=["accuracy"])
56. model.fit(x_treino, y_treino, batch_size=tamanho_batch, epochs=epocas,
57.           validation_split=0.1)
57.
58. resultados = model.evaluate(x_teste, y_teste, verbose=0)
59. print("perca, conjunto de teste:", resultados[0])
60. print("acuracia, conjunto de teste:", resultados[1])
61. model.save("exemploCNN")
```

Fonte: Teixeira, 2022.

5.2 Carregando o modelo salvo

Código 13: Carregando um modelo já treinado.

```
-----
1. import numpy as np
```



```
2. from tensorflow import keras
3. from tensorflow.keras import layers
4. import matplotlib.pyplot as plt

5. num_classes = 10
6. formato_entrada = (28, 28, 1)

7. # Carregando e separando os dados entre treino e teste
8. (x_treino, y_treino), (x_teste, y_teste) = keras.datasets.mnist.load_data()

9. #Verificando as dimensões dos dados
10. print("Dimensão de x_treino:", x_treino.shape)
11. print(x_treino.shape[0], "treino")
12. print(x_teste.shape[0], "teste")

13. ##Convertendo a saída de único número inteiro para vetor de ativação
14. y_treino = keras.utils.to_categorical(y_treino, num_classes)
15. y_teste = keras.utils.to_categorical(y_teste, num_classes)

16. #Carrega modelo da memória
17. model = keras.models.load_model("exemploCNN")

18. model.summary()

19. #Verifica a perda e a acurácia
20. resultados = model.evaluate(x_teste, y_teste, verbose=0)
21. print("perca, conjunto de teste:", resultados[0])
22. print("acuracia, conjunto de teste:", resultados[1])
```

Fonte: Teixeira, 2022.



FINALIZANDO

Nesta etapa, estudamos uma das principais ferramentas da *Deep Learning*, a Convolutional Neural Network (CNN). Começamos com conceitos básicos e sua principal diferença entre uma MLP tradicional, visto em conteúdo anterior, e então fomos para os principais tipos de camadas existentes em uma CNN, sendo elas a camada convolucional, a camada de pooling e a camada de rede totalmente conectada. Após vermos os conceitos, realizamos um exemplo onde criamos uma CNN do zero, definimos a estrutura da rede, e então realizamos o treinamento. Além de treinar a rede, a salvamos em nosso computador e a carregamos e executamos novamente, sem a necessidade de treinar novamente a rede. Esta etapa cumpriu com todos os seus objetivos propostos, dando a você um conceito básico sobre CNN e seu funcionamento, por meio da biblioteca Keras e tensorflow.



REFERÊNCIAS

CHO, Y.; SAUL, L. Kernel methods for deep learning. **Advances in neural information processing systems**, v. 22, 2009.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep learning**. MIT press, 2016.

GULLI, A.; PAL, S. **Deep learning with Keras**. Packt Publishing Ltd, 2017.

HAO, X.; ZHANG, G.; MA, S. Deep learning. **International Journal of Semantic Computing**, v. 10, n. 3, p. 417-439, 2016.

LECUN, Y.; BOTTOU, L.; BENGIO, Y.; HAFFNER, P. **Gradient-based learning applied to document recognition**. Proc. of the IEEE, 1998. Disponível em: <<http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf>>. Acesso em: 18 ago. 2022.